

School of Computer Science and Engineering

Fall Semester 2026-27

BCSE331P – Exploratory Data Analysis

Lab Assessment -1

Name : Arjun Mishra

Reg no : 23BDS0334

Lab : L31+L32

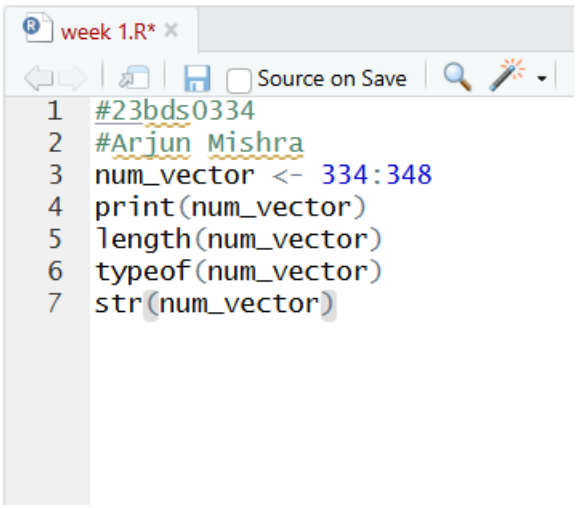
R data structures and data types

1. Create a numeric vector with 15 elements and display its length, type, and structure.

Aim

To create a numeric vector with 15 elements and display its length, type, and structure.

Code:



Output:

```
> #23bds0334
> #Arjun Mishra
> num_vector <- 334:348
> print(num_vector)
 [1] 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348
> length(num_vector)
[1] 15
> typeof(num_vector)
[1] "integer"
> str(num_vector)
 int [1:15] 334 335 336 337 338 339 340 341 342 343 ...
>
```

Result:

A numeric vector with 15 elements was created successfully, and its length, type, and structure were displayed.

2.Create a character vector containing 10 student names and extract the 3rd, 5th, and 8th names.

Aim:

To create a character vector containing 10 student names and extract the 3rd, 5th, and 8th names.

Code:



Output:



Result:

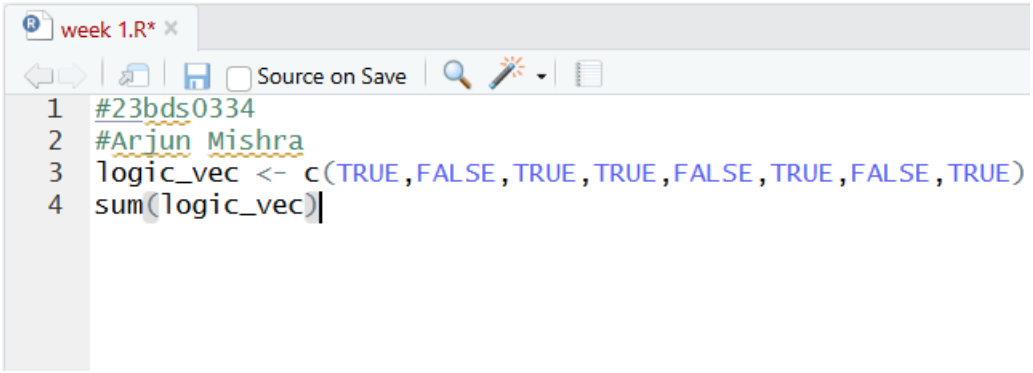
The required student names were extracted successfully.

3. Create a logical vector with TRUE and FALSE values. Count how many TRUE values are present.

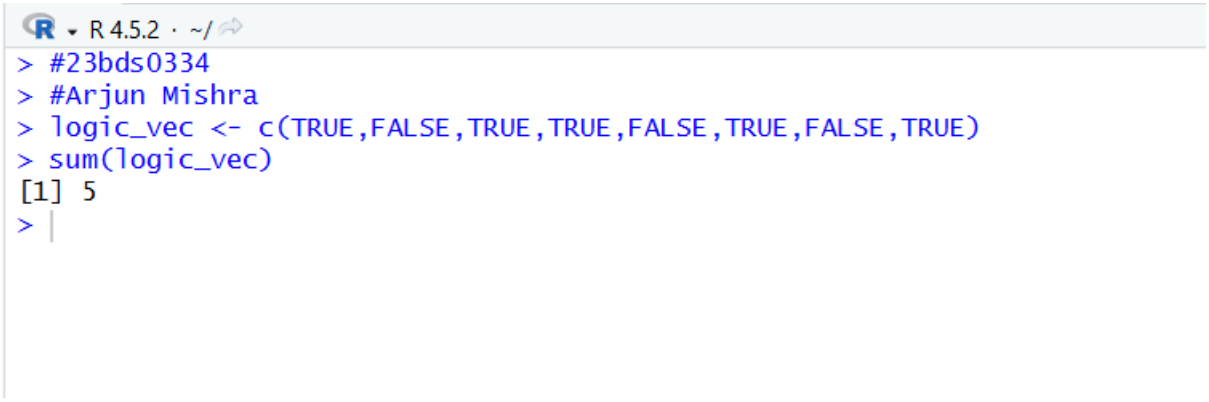
Aim:

To create a logical vector and count the number of TRUE values.

Code:



Output:



Result:

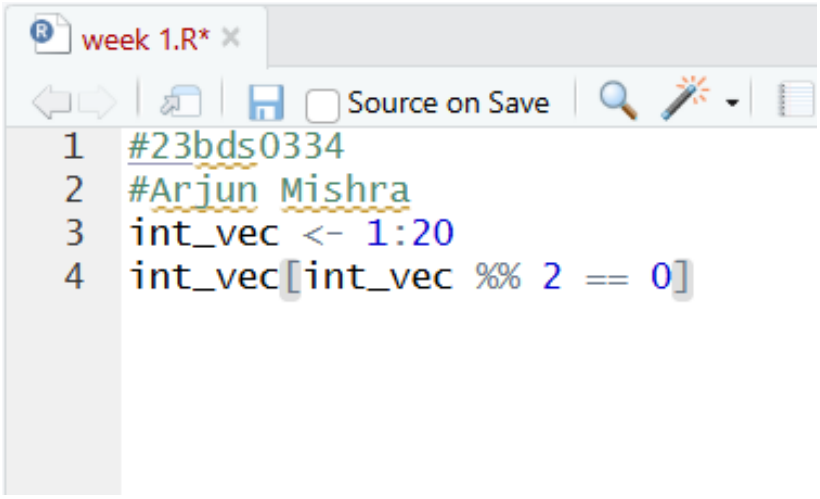
The number of TRUE values was counted successfully.

4.Create an integer vector from 1 to 20 and extract only the even numbers.

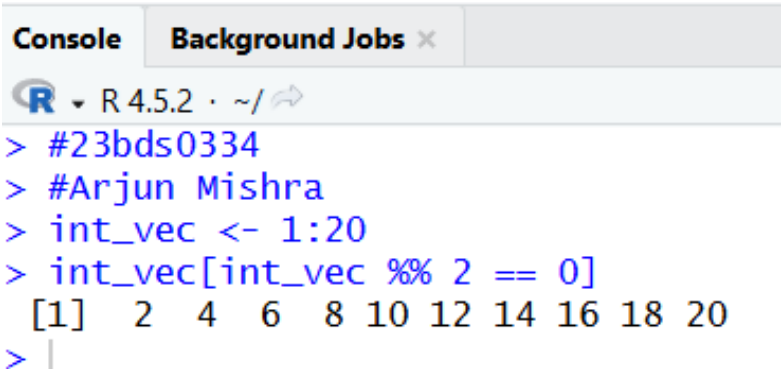
Aim:

To create an integer vector from 1 to 20 and extract only the even numbers.

Code:



Output:



Result:

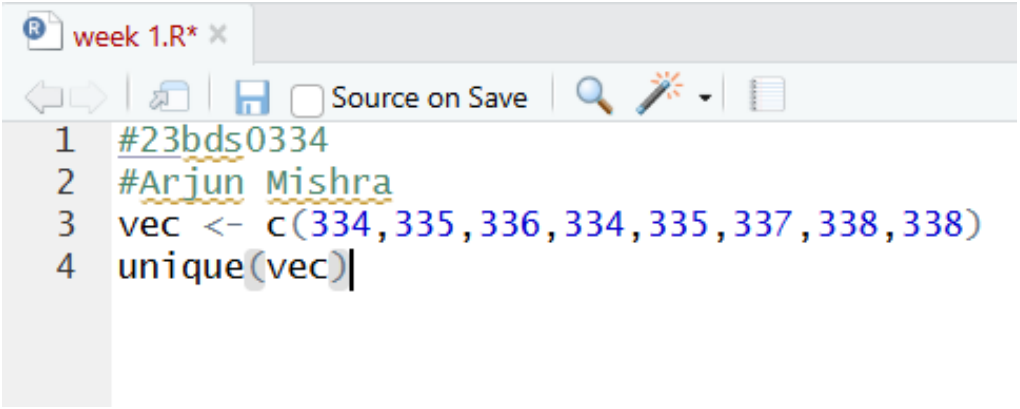
The even numbers were extracted successfully from the integer vector.

5.Create a vector with repeated values and find the unique values using an R function.

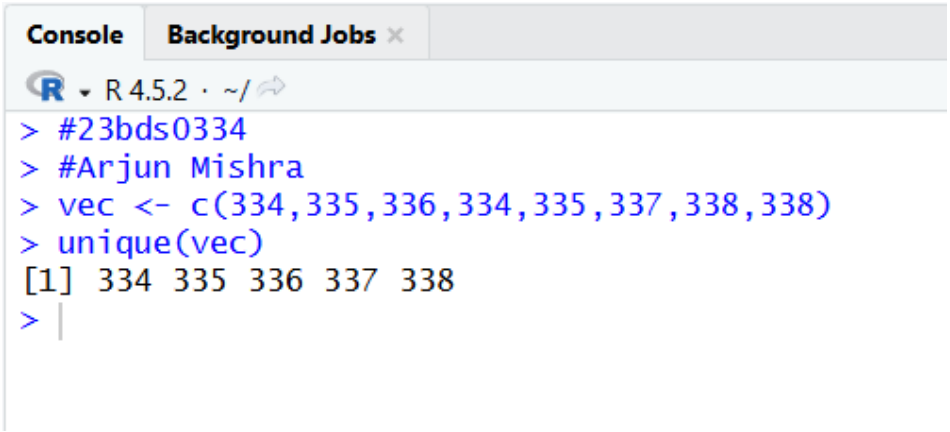
Aim:

To create a vector with repeated values and find the unique values.

Codes :



Output:



Result:

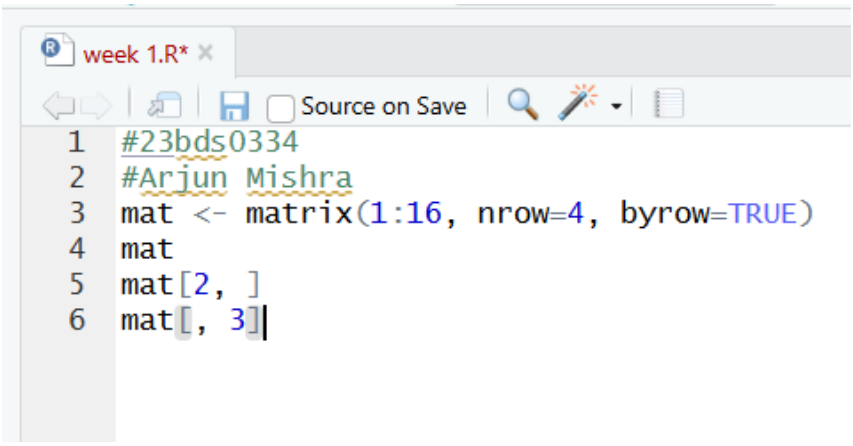
The unique values in the vector were displayed successfully.

6.Create a 4 × 4 matrix using numbers from 1 to 16 and display the second row and third column.

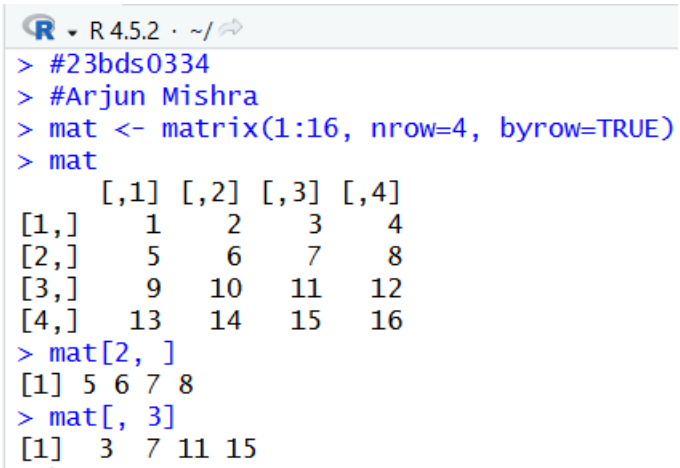
Aim:

To create a 4 × 4 matrix using numbers from 1 to 16 and display the second row and third column.

Code:



Output:



Result:

A 4 × 4 matrix was created successfully, and the second row and third column were displayed.

7.Create two matrices of the same size and perform element-wise addition, subtraction, multiplication, and division.

Aim:

To create two matrices of the same size and perform element-wise addition, subtraction, multiplication, and division.

Code:

week 1.R*

Source on Save

```
#23bds0334
#Arjun Mishra
A <- matrix(334:342, nrow=3)
B <- matrix(343:351, nrow=3)

A + B
A - B
A * B
A / B
```

Output:

Console | Background Jobs

R 4.5.2 · ~/

```
> #23bds0334
> #Arjun Mishra
> A <- matrix(334:342, nrow=3)
> B <- matrix(343:351, nrow=3)
>
> A + B
     [,1] [,2] [,3]
[1,]  677  683  689
[2,]  679  685  691
[3,]  681  687  693
> A - B
     [,1] [,2] [,3]
[1,]   -9   -9   -9
[2,]   -9   -9   -9
[3,]   -9   -9   -9
> A * B
       [,1]   [,2]   [,3]
[1,] 114562 116602 118660
[2,] 115240 117286 119350
[3,] 115920 117972 120042
> A / B
          [,1]      [,2]      [,3]
[1,] 0.9737609 0.9739884 0.9742120
[2,] 0.9738372 0.9740634 0.9742857
[3,] 0.9739130 0.9741379 0.9743590
>
```

Result:

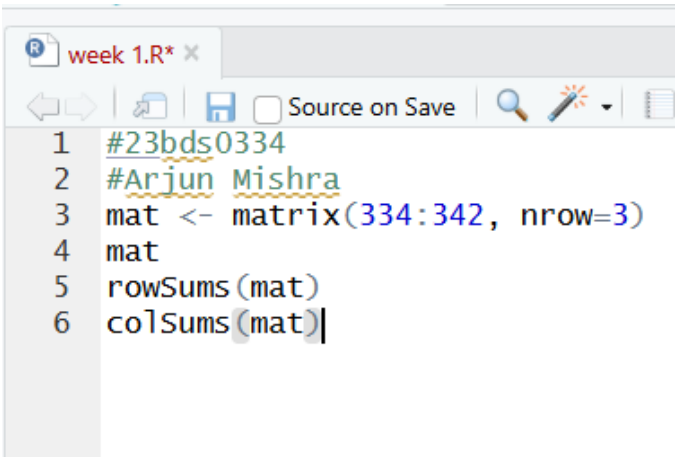
Element-wise arithmetic operations were performed successfully on the two matrices.

8.Create a 3 × 3 matrix and find the row sums and column sums.

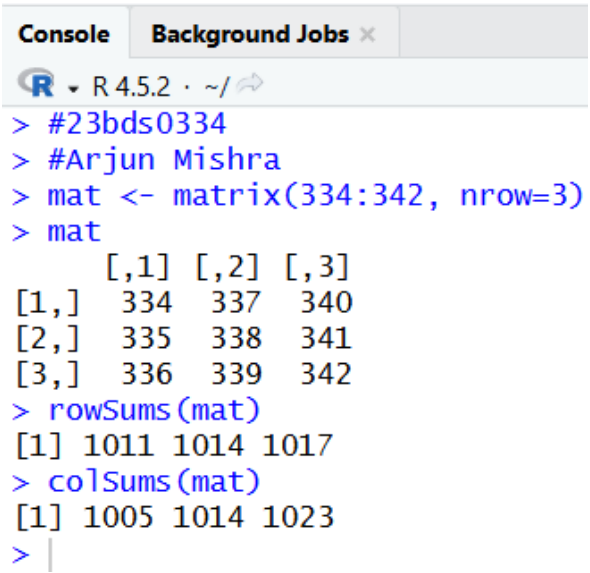
Aim:

To create a 3 × 3 matrix and find the row sums and column sums.

Code:



Output:



Result:

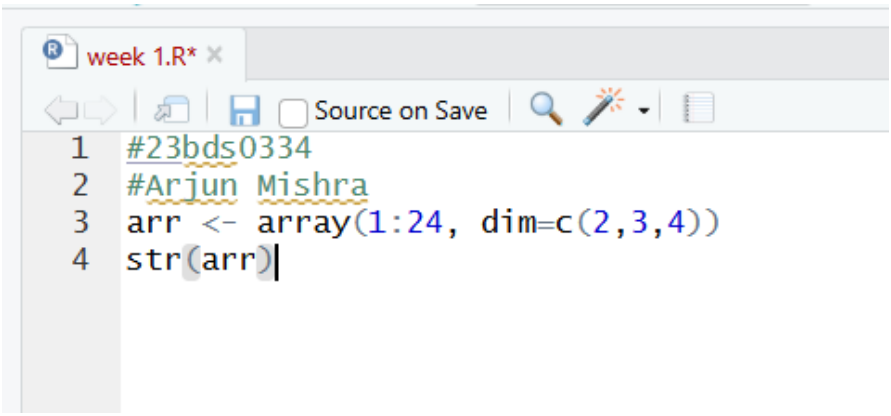
The row sums and column sums of the matrix were calculated successfully.

9.Create a 3-dimensional array with dimensions 2 × 3 × 4 and display its structure.

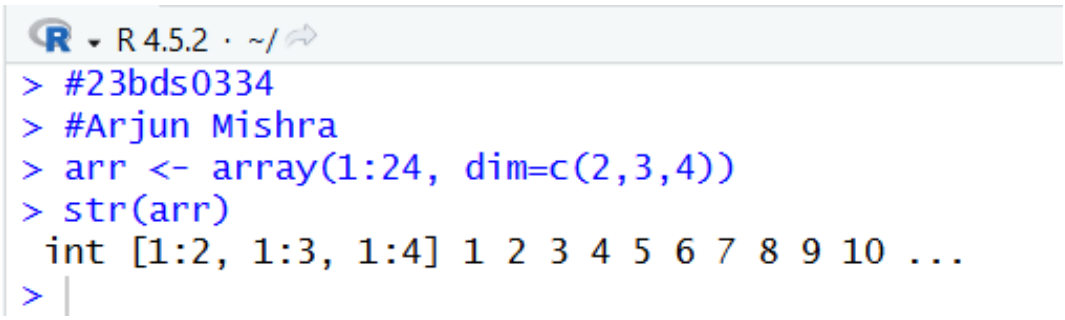
Aim:

To create a three-dimensional array with dimensions 2 × 3 × 4 and display its structure.

Code:



Output:



Result :

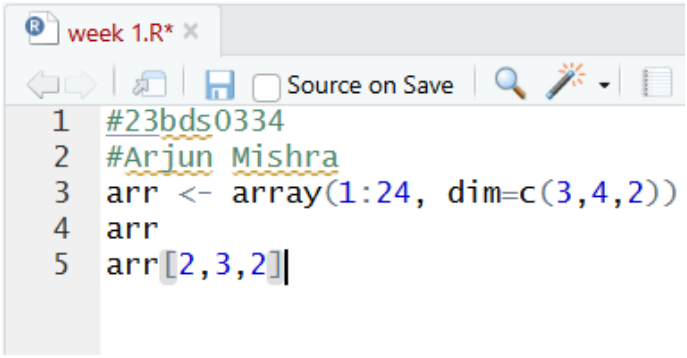
A three-dimensional array was created successfully, and its structure was displayed.

10.Create an array using numbers from 1 to 24 with dimensions 3 × 4 × 2 and extract one specific element.

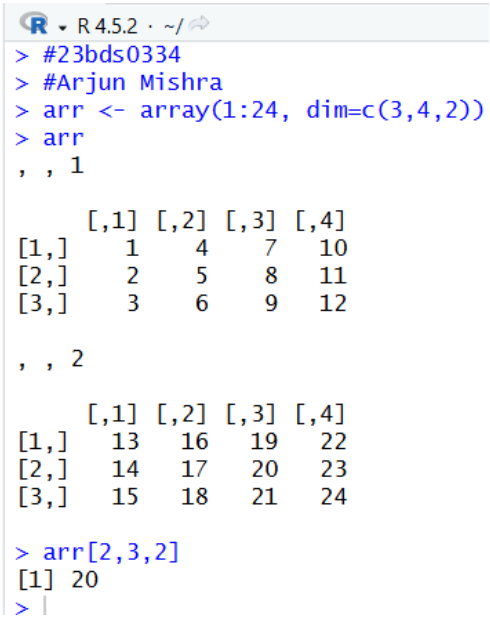
Aim:

To create an array using numbers from 1 to 24 with dimensions 3 × 4 × 2 and extract one specific element.

Code:



Output:



Result:

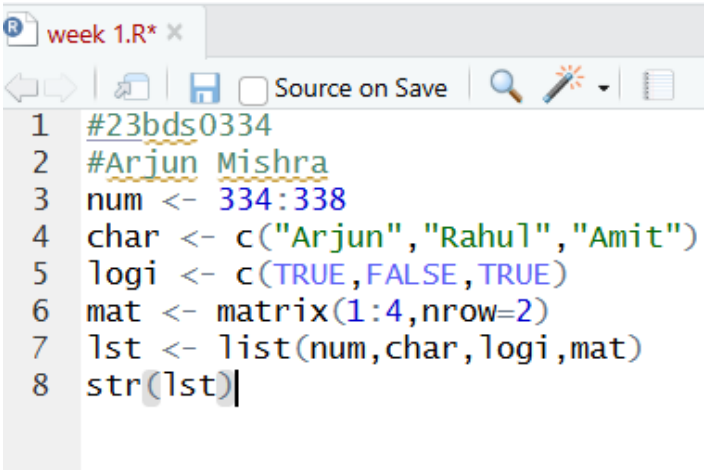
The required element was extracted successfully from the array.

11.Create a list containing a numeric vector, character vector, logical vector, and matrix. Display the list structure.

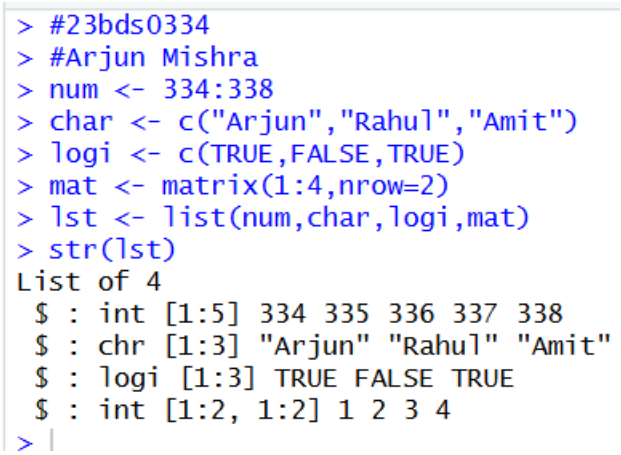
Aim:

To create a list containing a numeric vector, character vector, logical vector, and matrix, and display its structure.

Code:



Output:



Result:

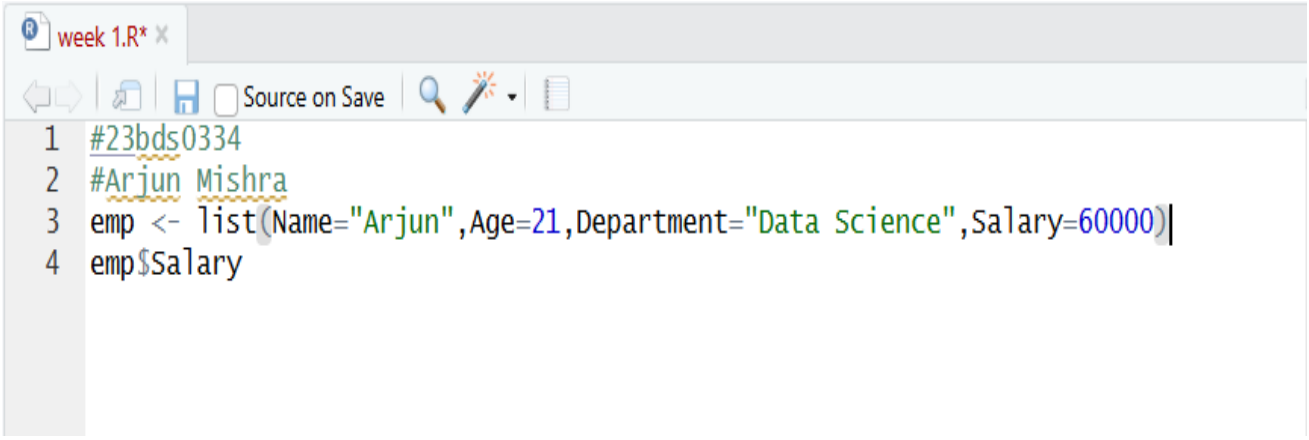
A list containing different data types was created successfully, and its structure was displayed.

12.Create a named list with elements Name, Age, Department, and Salary. Access the Salary element using the $ operator.

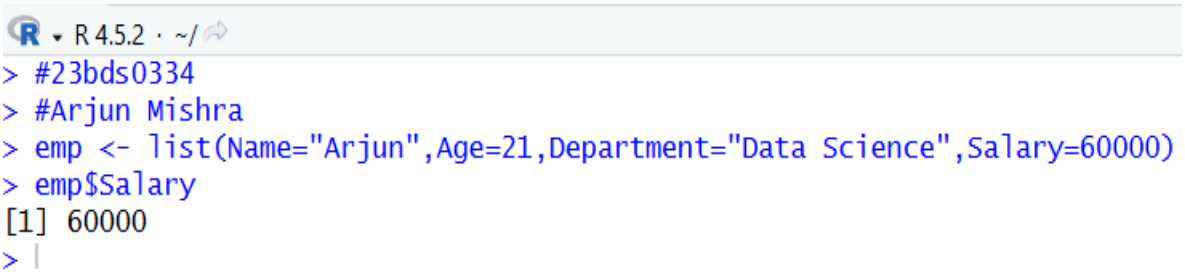
Aim:

To create a named list with Name, Age, Department, and Salary, and access the Salary element.

Code:



Output:



Result:

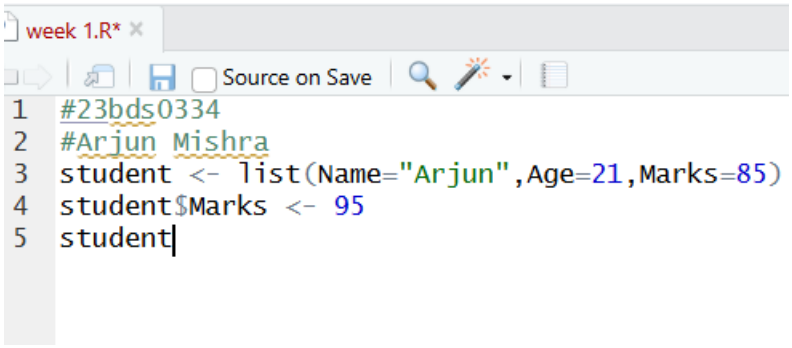
The Salary element was accessed successfully using the $ operator.

13 Create a list of student details and update one element in the list.

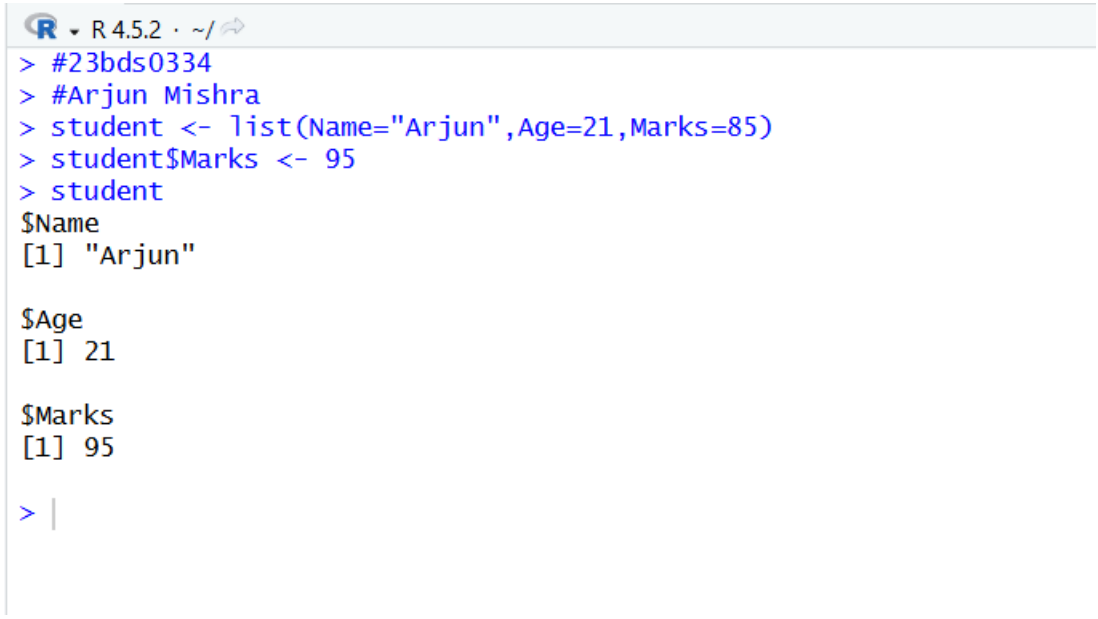
Aim:

To create a list of student details and update one element in the list.

Code:



Output:



Result:

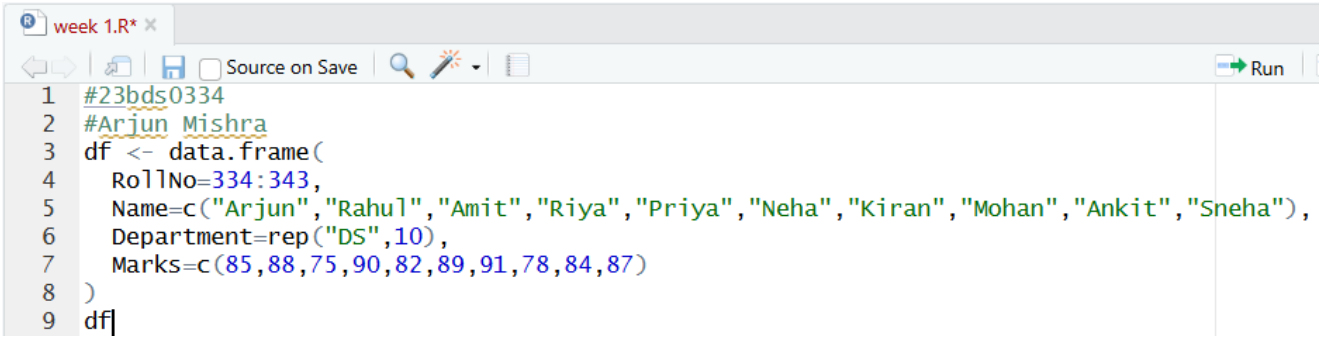
The student list was updated successfully.

14.Create a dataframe with columns RollNo, Name, Department, and Marks for 10 students.

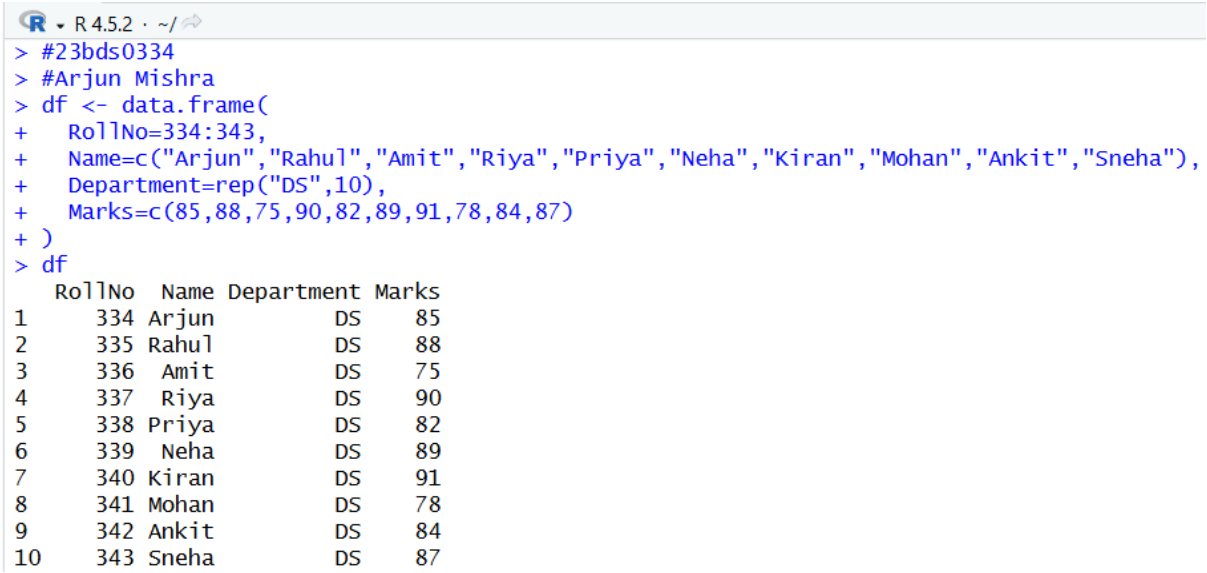
Aim:

To create a dataframe with RollNo, Name, Department, and Marks for 10 students.

Code:



Output:



Result:

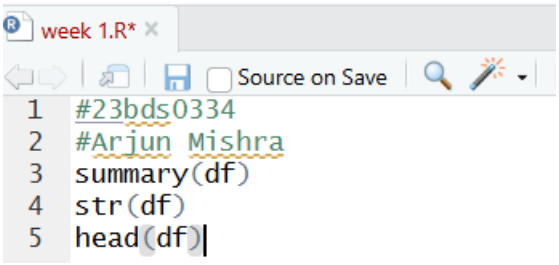
A dataframe containing details of 10 students was created successfully.

15. Display the summary, structure, and first six rows of a dataframe.

Aim:

To display the summary, structure, and first six rows of a dataframe.

Code:



Output:

```
> #23bds0334
> #Arjun Mishra
> summary(df)
     RollNo          Name            Department            Marks
 Min.   :334.0   Length:10          Length:10          Min.   :75.00
 1st Qu.:336.2   Class :character   Class :character   1st Qu.:82.50
 Median :338.5   Mode  :character   Mode  :character   Median :86.00
 Mean   :338.5                                         Mean   :84.90
 3rd Qu.:340.8                                         3rd Qu.:88.75
 Max.   :343.0                                         Max.   :91.00
> str(df)
'data.frame':   10 obs. of  4 variables:
 $ RollNo    : int  334 335 336 337 338 339 340 341 342 343
 $ Name      : chr  "Arjun" "Rahul" "Amit" "Riya" ...
 $ Department: chr  "DS" "DS" "DS" "DS" ...
 $ Marks     : num  85 88 75 90 82 89 91 78 84 87
> head(df)
  RollNo  Name Department Marks
1    334 Arjun         DS    85
2    335 Rahul         DS    88
3    336  Amit         DS    75
4    337  Riya         DS    90
5    338 Priya         DS    82
6    339  Neha         DS    89
>
```

Result:

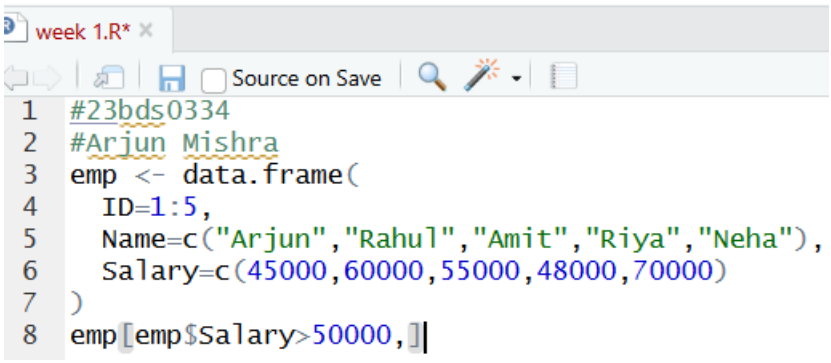
The summary, structure, and first six rows of the dataframe were displayed successfully.

16.Create a dataframe of employee details and extract only employees whose salary is greater than 50,000.

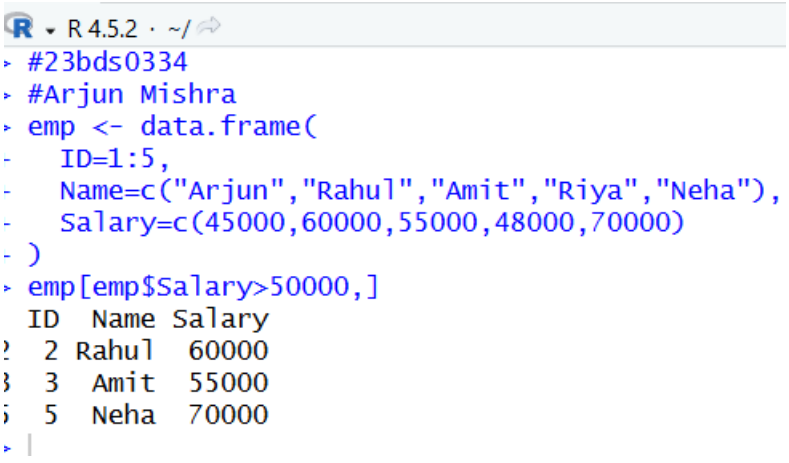
Aim:

To create a dataframe of employee details and extract employees whose salary is greater than 50,000.

Code:



Output:



Result:

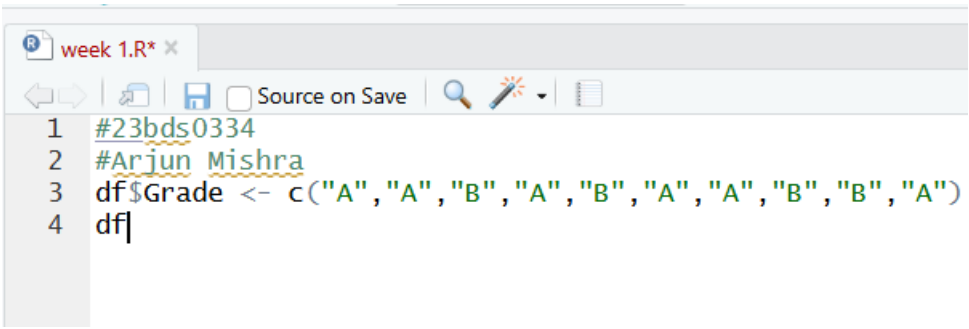
Employees with salary greater than ₹50,000 were displayed successfully.

17.Add a new column called Grade to an existing student dataframe.

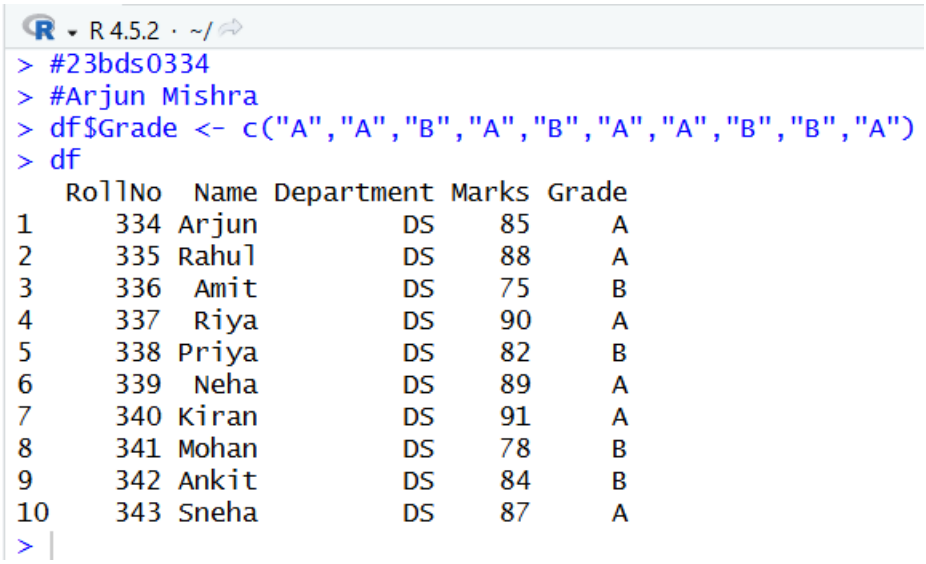
Aim:

To add a new column called **Grade** to an existing student dataframe.

Code:



Output:



Result:

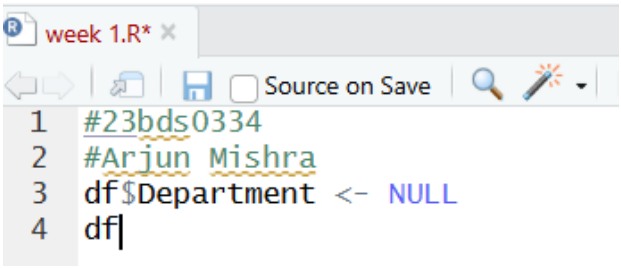
A new column **Grade** was added successfully to the dataframe.

18.Remove one column from a dataframe and display the updated dataframe.

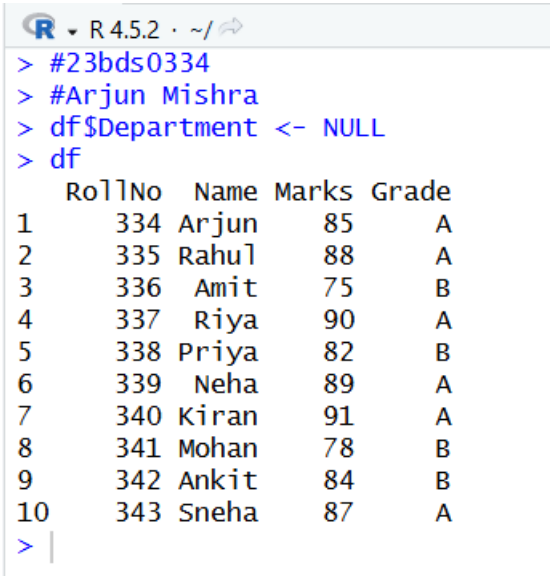
Aim:

To remove one column from a dataframe and display the updated dataframe.

Code:



Output:



Result:

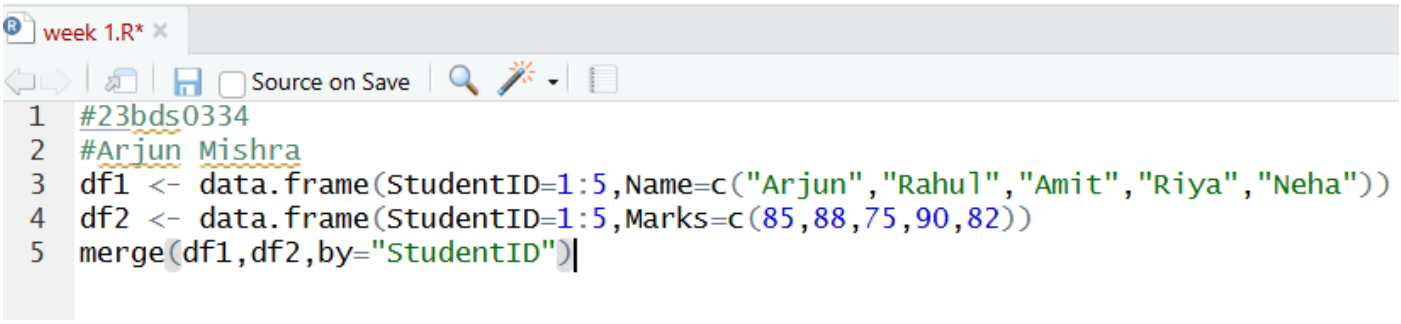
The Department column was removed successfully from the dataframe.

19.Create two dataframes with a common column StudentID and merge them using that column.

Aim:

To create two dataframes with a common column StudentID and merge them.

Code:



Output:

```
> #23bds0334
> #Arjun Mishra
> df1 <- data.frame(StudentID=1:5,Name=c("Arjun","Rahul","Amit","Riya","Neha"))
> df2 <- data.frame(StudentID=1:5,Marks=c(85,88,75,90,82))
> merge(df1,df2,by="StudentID")
  StudentID  Name Marks
1         1 Arjun    85
2         2 Rahul    88
3         3  Amit    75
4         4  Riya    90
5         5  Neha    82
>
```

Result:

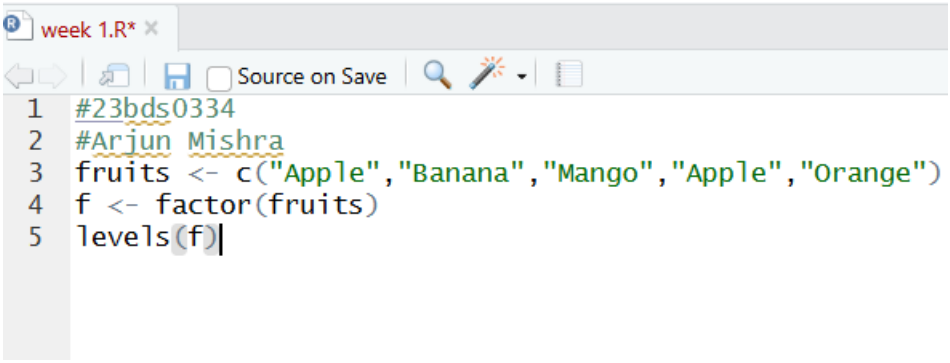
The two dataframes were merged successfully using StudentID.

20. Convert a character vector of fruit names into a factor and display its levels.

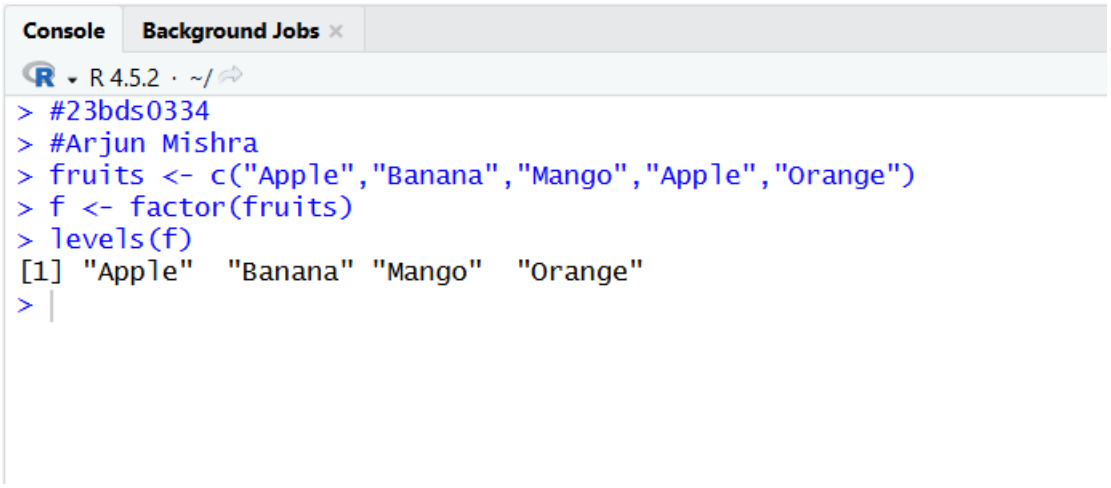
Aim:

To convert a character vector of fruit names into a factor and display its levels.

Code:



Output:



Result:

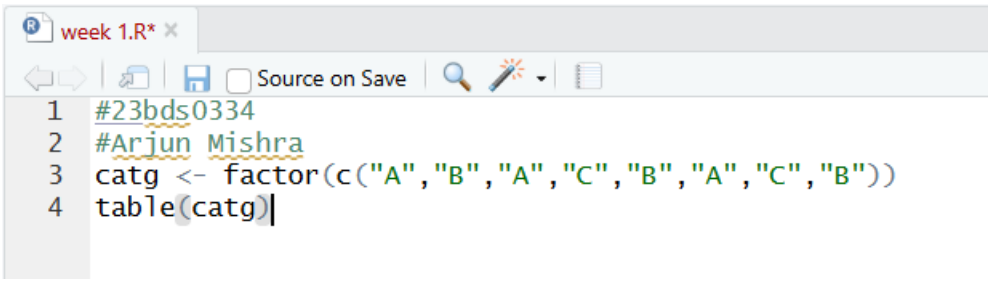
The character vector was converted into a factor, and its levels were displayed successfully.

21. Create a factor with repeated categories and count the frequency of each level.

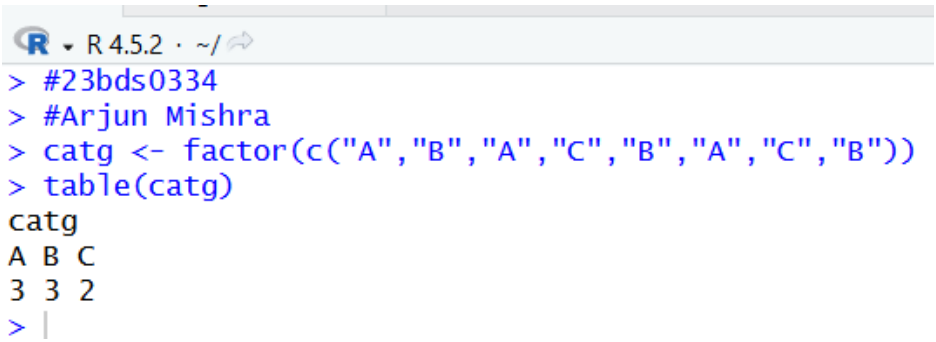
Aim:

To create a factor with repeated categories and count the frequency of each level.

Code:



Output:



Result:

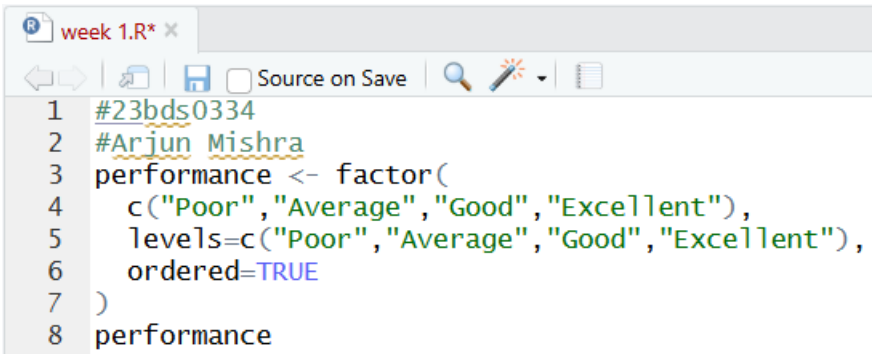
The factor was created successfully, and the frequency of each level was displayed.

22.Create an ordered factor for student performance levels: Poor, Average, Good, Excellent.

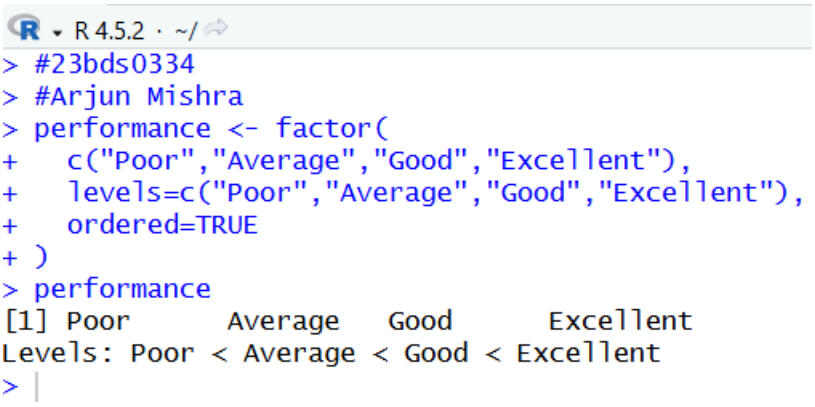
Aim:

To create an ordered factor for student performance levels: Poor, Average, Good, and Excellent.

Code:



Output:



Result:

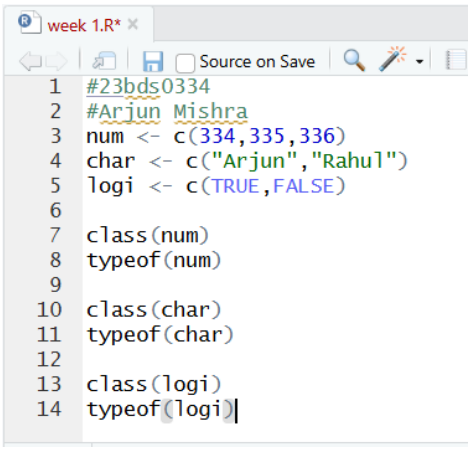
An ordered factor for student performance levels was created successfully.

23. Create three vectors: numeric, character, and logical. Display their class and data type.

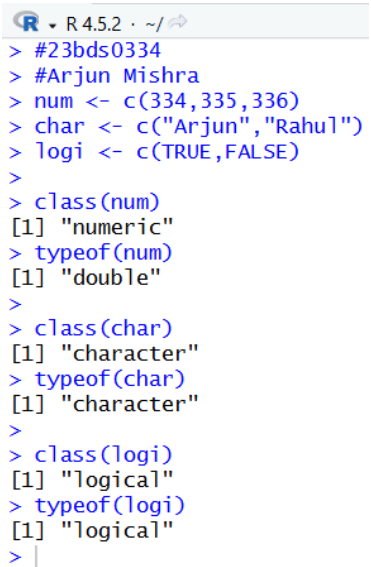
Aim:

To create numeric, character, and logical vectors and display their class and data type.

Code:



Output:



Result:

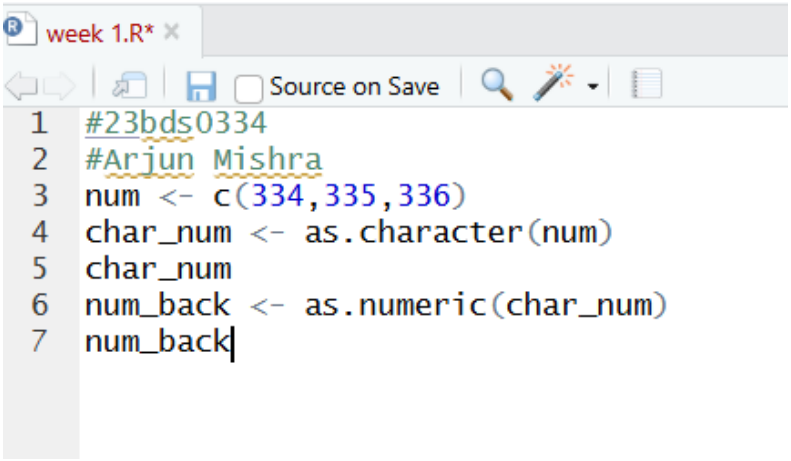
The class and data type of numeric, character, and logical vectors were displayed successfully.

24.Convert a numeric vector into character type and then convert it back to numeric type.

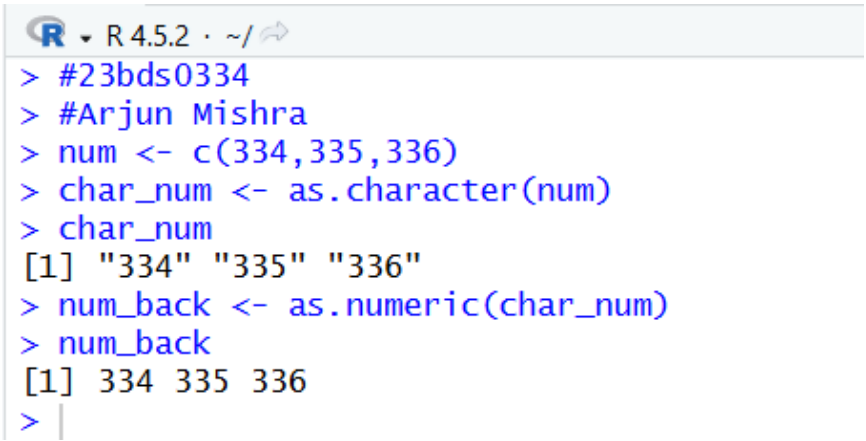
Aim:

To convert a numeric vector into character type and then convert it back to numeric type.

Code:



Output:



Result:

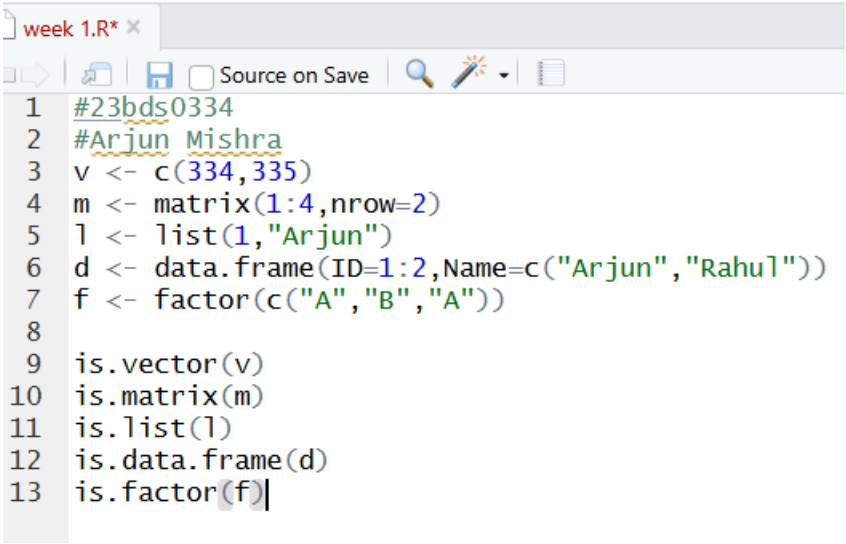
The numeric vector was successfully converted to character type and then back to numeric type.

25.Write an R program to check whether a given object is a vector, matrix, list, dataframe, or factor using suitable R functions.

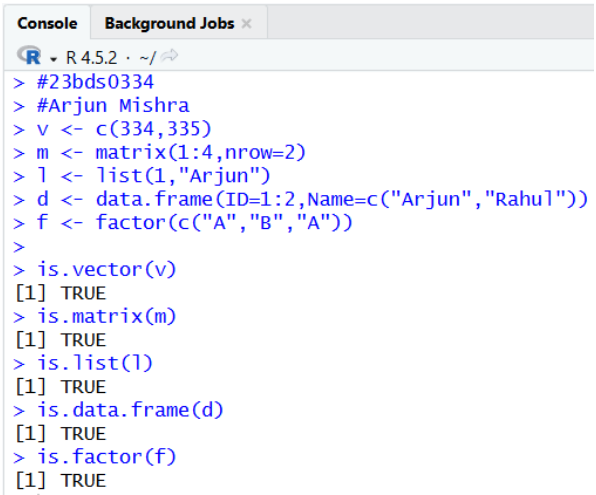
Aim:

To check whether a given object is a vector, matrix, list, dataframe, or factor using suitable R functions.

Code:



Output:



Result:

The object types were identified successfully using the appropriate R functions.

WEEK 2

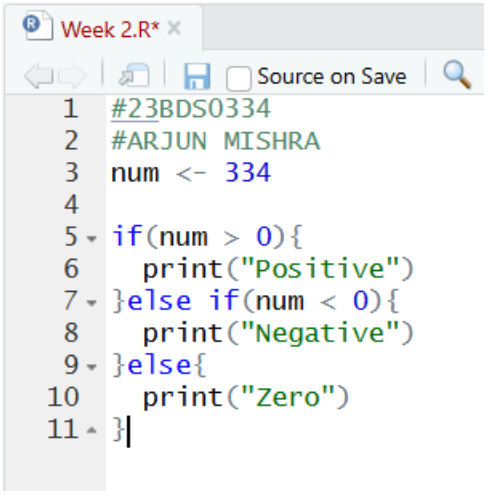
Branching, Looping, and Functions in R Programming

1. Write an R program using an if-else statement to determine whether a given number is positive, negative, or zero.

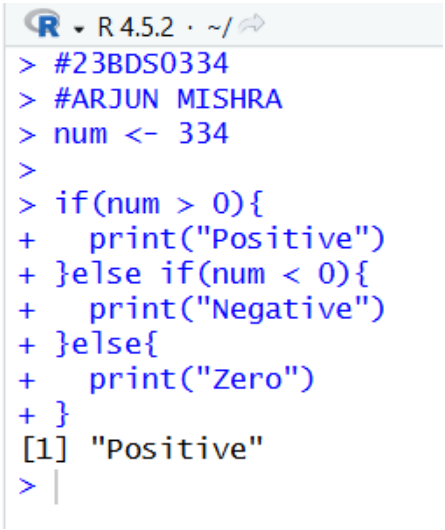
AIM:

To determine whether a given number is positive, negative, or zero using an if-else statement.

CODE:



OUTPUT:



RESULT:

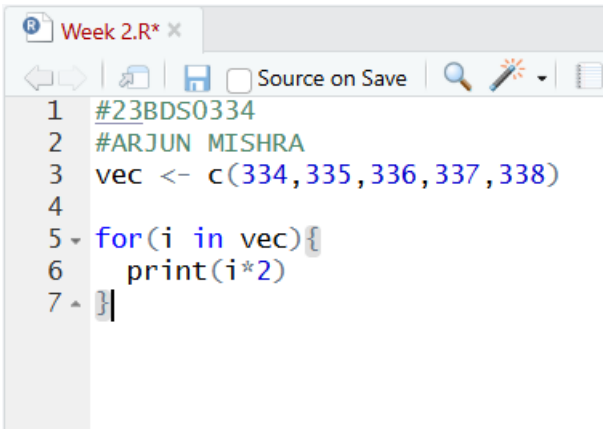
The given number was identified successfully as positive.

2. Create a for loop that iterates through a vector of numbers and prints twice the value of each element.

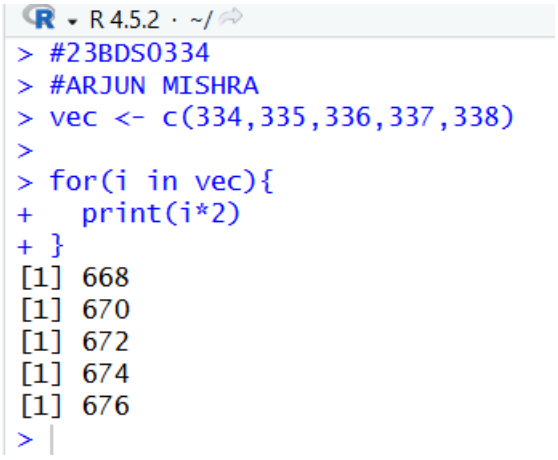
AIM:

To create a for loop that prints twice the value of each element in a vector.

CODE:



OUTPUT:



RESULT:

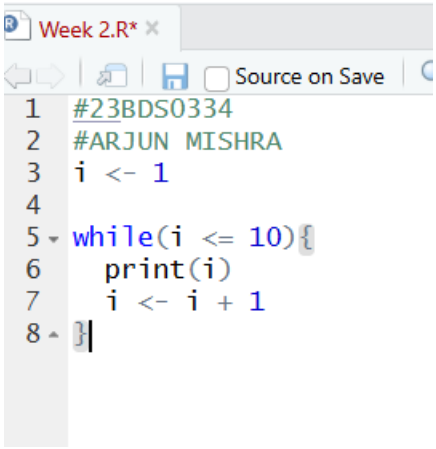
The for loop successfully printed twice the value of each vector element.

3. Write an R program using a while loop to print the numbers from 1 to 10. Explain the condition that terminates the loop.

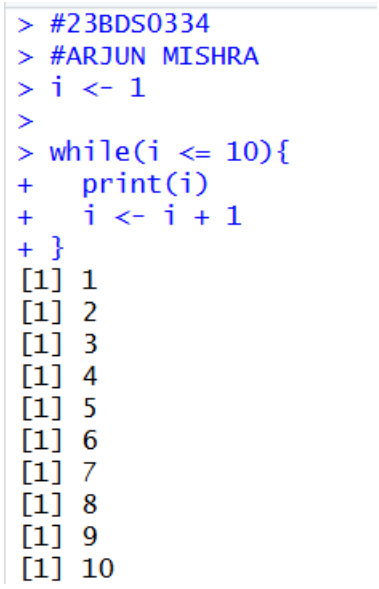
AIM:

To print the numbers from 1 to 10 using a while loop.

CODE:



OUTPUT:



RESULT:

The numbers from 1 to 10 were printed successfully. The loop terminated when i became greater than 10.

4. Use a repeat loop to print the message "Learning R" five times. Include an appropriate condition and a break statement to terminate the loop.

AIM:

To print the message "Learning R" five times using a repeat loop.

CODE:

Week 2.R*

Source on Save

```
#23BDS0334
#ARJUN MISHRA
count <- 1

repeat{
  print("Learning R")
  count <- count + 1

  if(count > 5){
    break
  }
}
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #23BDS0334
> #ARJUN MISHRA
> count <- 1
>
> repeat{
+   print("Learning R")
+   count <- count + 1
+
+   if(count > 5){
+     break
+   }
+ }
[1] "Learning R"
[1] "Learning R"
[1] "Learning R"
[1] "Learning R"
[1] "Learning R"
>
```

RESULT:

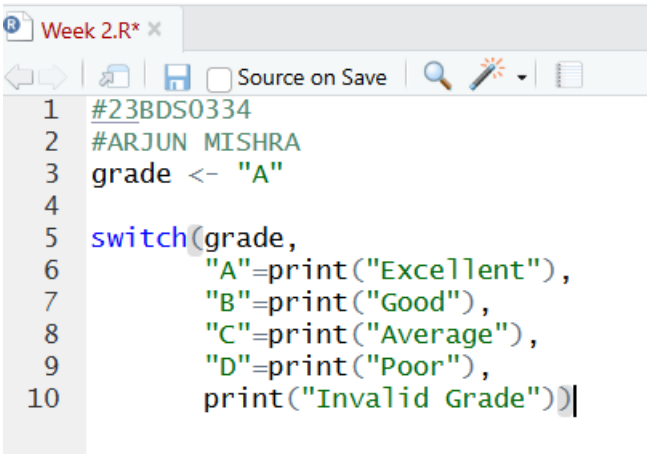
The message was printed five times successfully using a repeat loop.

5. Write an R program using a switch statement that accepts a grade such as "A", "B", "C", or "D" and displays "Excellent", "Good", "Average", or "Poor", respectively.

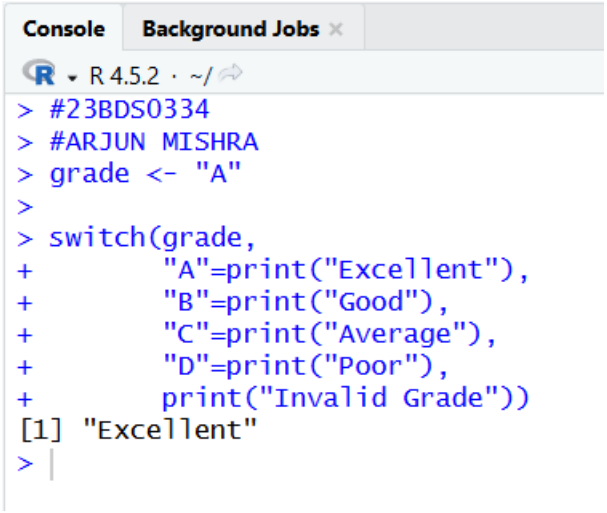
AIM:

To display the performance based on the given grade using a switch statement.

CODE:



OUTPUT:



RESULT:

The appropriate performance message was displayed successfully using the switch statement.

6. Create a nested for loop to generate multiplication tables for the numbers 1 to 5, with each table containing multiples from 1 to 10.

AIM:

To generate multiplication tables for numbers 1 to 5 using nested for loops.

CODE:

Week 2.R* | Source on Save

```
#23BDS0334
#ARJUN MISHRA
for(i in 1:5){
  cat("Table of",i,"\n")
  for(j in 1:10){
    cat(i,"x",j,"=",i*j,"\n")
  }
  cat("\n")
}
```

OUTPUT:

```
> #23BDS0334
> #ARJUN MISHRA
> for(i in 1:5){
+   cat("Table of",i,"\n")
+   for(j in 1:10){
+     cat(i,"x",j,"=",i*j,"\n")
+   }
+   cat("\n")
+ }
Table of 1
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10

Table of 2
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20

Table of 3
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
```

```
Table of 4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40

Table of 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

>
```

RESULT:

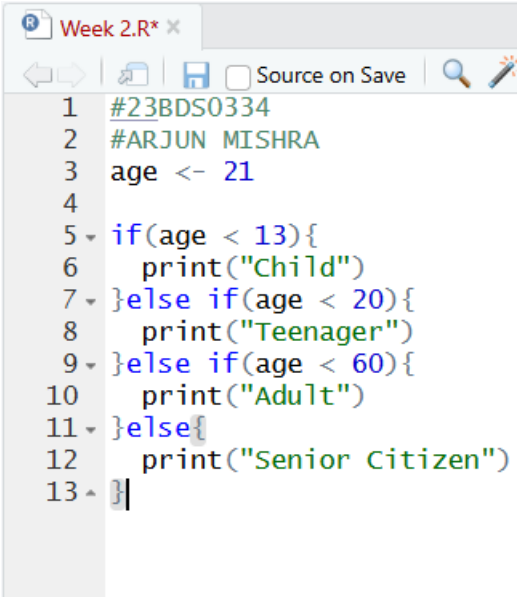
Multiplication tables from 1 to 5 were generated successfully.

7. Write an R program using conditional statements to classify a person as a "Child", "Teenager", "Adult", or "Senior Citizen" based on their age.

AIM:

To classify a person as Child, Teenager, Adult, or Senior Citizen based on age.

CODE:



OUTPUT:

```
> #23BDS0334
> #ARJUN MISHRA
> age <- 21
>
> if(age < 13){
+   print("Child")
+ }else if(age < 20){
+   print("Teenager")
+ }else if(age < 60){
+   print("Adult")
+ }else{
+   print("Senior Citizen")
+ }
[1] "Adult"
>
```

RESULT:

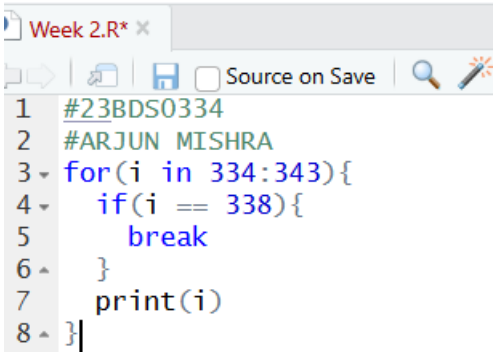
The person was classified successfully based on age.

8. Demonstrate the use of the break statement in a for loop by terminating the loop when a specified value is encountered.

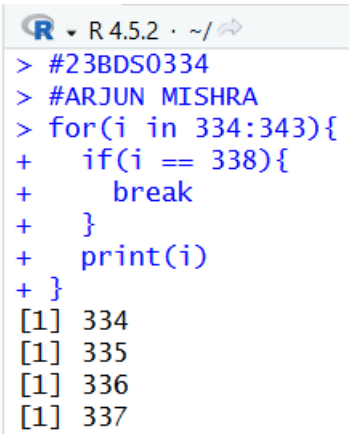
AIM:

To demonstrate the use of the break statement in a for loop.

CODE:



OUTPUT:



RESULT:

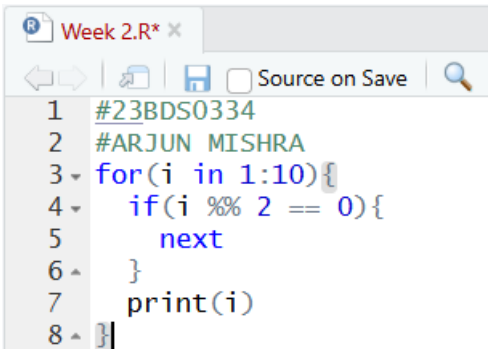
The loop terminated successfully when the specified value was encountered.

9. Write an R program using the next statement to skip even numbers and print only the odd numbers between 1 and 10.

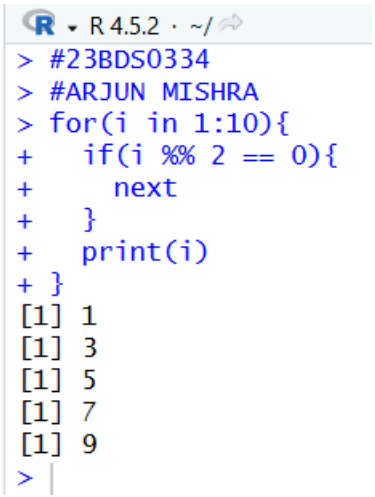
AIM:

To print only odd numbers between 1 and 10 using the next statement.

CODE:



OUTPUT:



RESULT:

Only the odd numbers were printed successfully by skipping even numbers.

10. Create a for loop to calculate the factorial of a given non-negative integer.

AIM:

To calculate the factorial of a given non-negative integer using a for loop.

CODE:

Week 2.R* ×

Source on Save

```
1  #23BDS0334
2  #ARJUN MISHRA
3  num <- 5
4  fact <- 1
5
6  for(i in 1:num){
7    fact <- fact * i
8  }
9
10 print(fact)
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ARJUN MISHRA
> num <- 5
> fact <- 1
>
> for(i in 1:num){
+   fact <- fact * i
+ }
>
> print(fact)
[1] 120
>
```

RESULT:

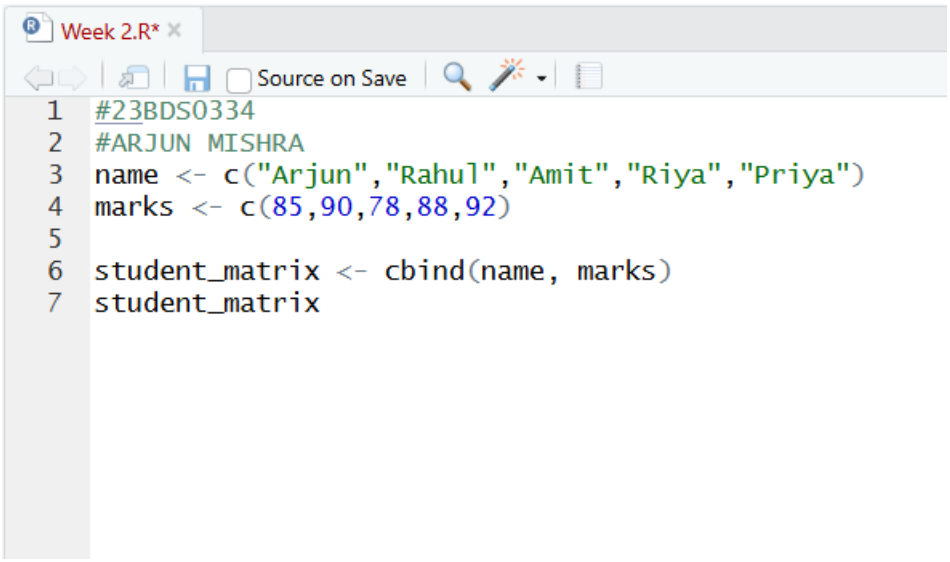
The factorial of the given number was calculated successfully.

11.Create two vectors containing student names and marks. Use the cbind() function to combine them column-wise into a matrix.

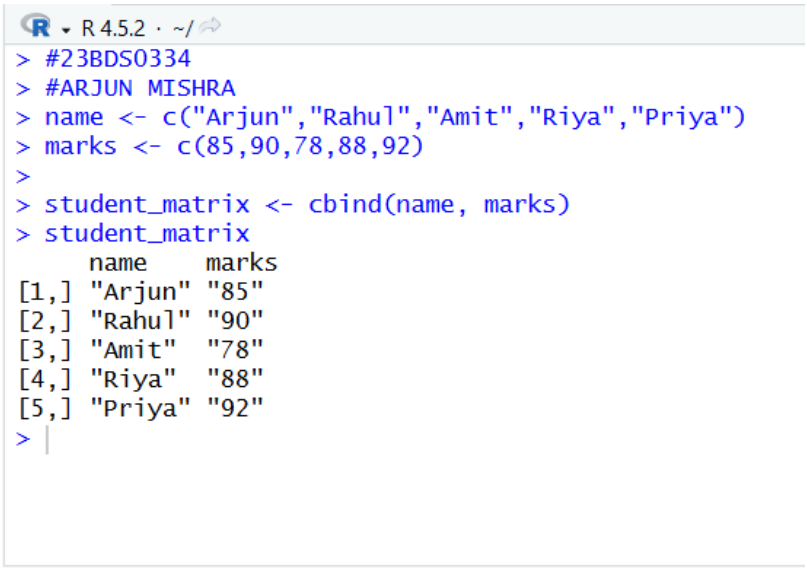
AIM:

To create two vectors containing student names and marks and combine them column-wise into a matrix using cbind().

CODE:



OUTPUT:



RESULT:

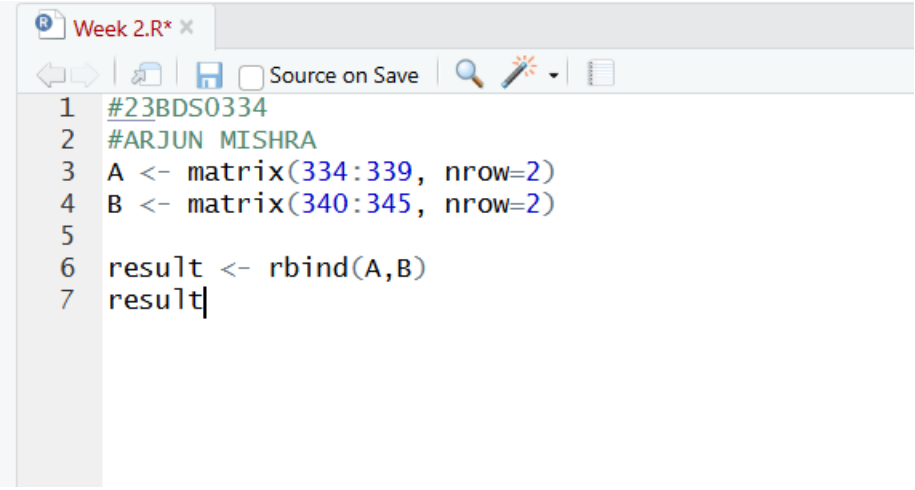
The two vectors were combined successfully into a matrix using cbind().

12.Create two matrices with the same number of columns and combine them row-wise using the rbind() function. Display the resulting matrix.

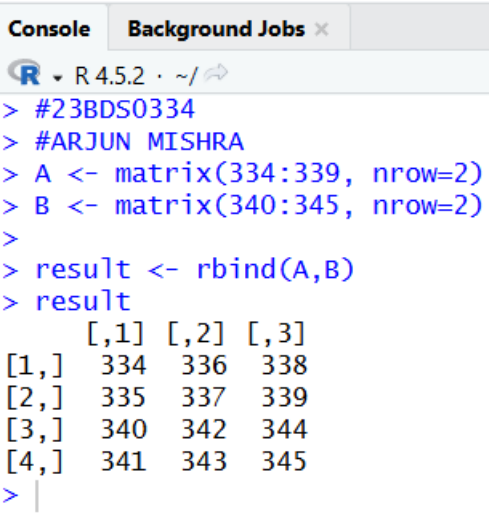
AIM:

To create two matrices and combine them row-wise using rbind().

CODE:



OUTPUT:



RESULT:

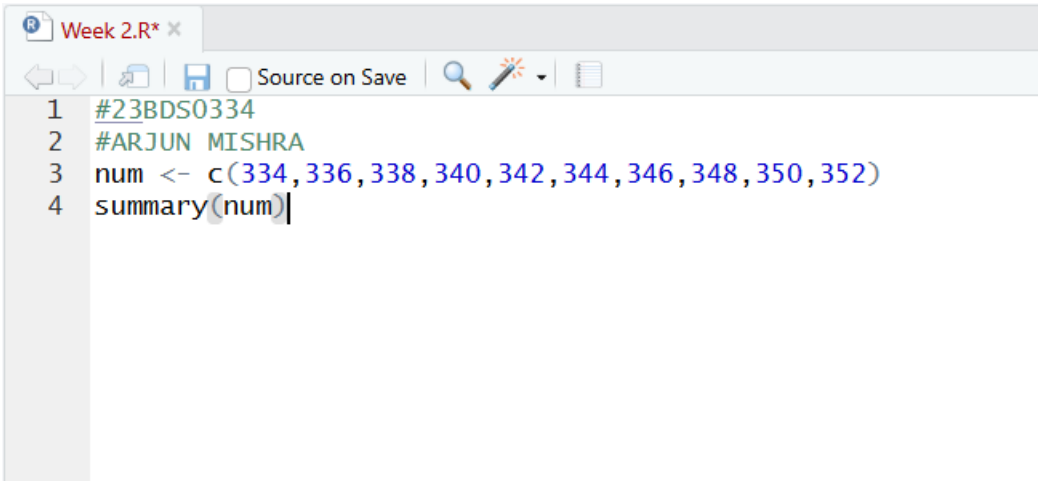
The two matrices were combined successfully using rbind().

13. Create a numeric vector containing at least ten observations and use the summary() function to display its minimum, first quartile, median, mean, third quartile, and maximum values.

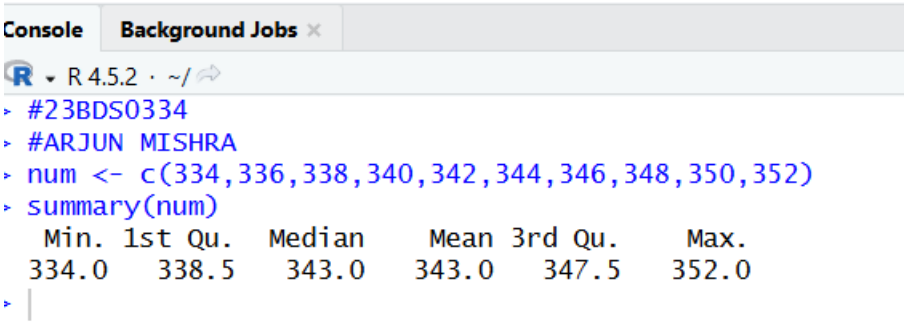
AIM:

To create a numeric vector and display its statistical summary using summary().

CODE:



OUTPUT:



RESULT:

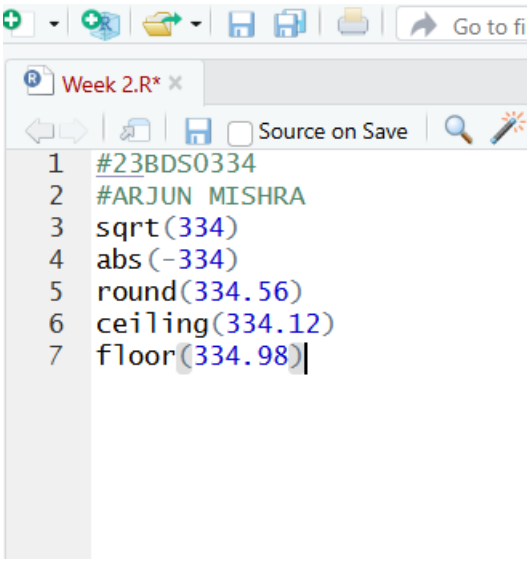
The summary statistics of the numeric vector were displayed successfully.

14. Write an R program that demonstrates any five built-in mathematical functions, such as sqrt(), abs(), round(), ceiling(), floor(), log(), or exp().

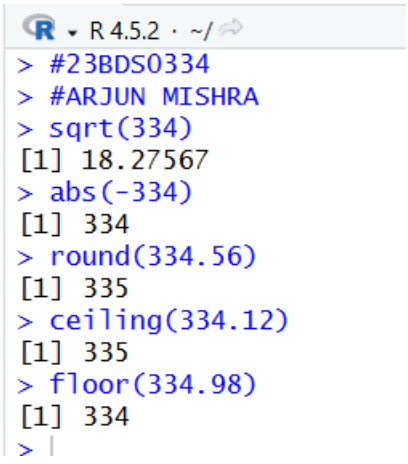
AIM:

To demonstrate the use of five built-in mathematical functions.

CODE:



OUTPUT:



RESULT:

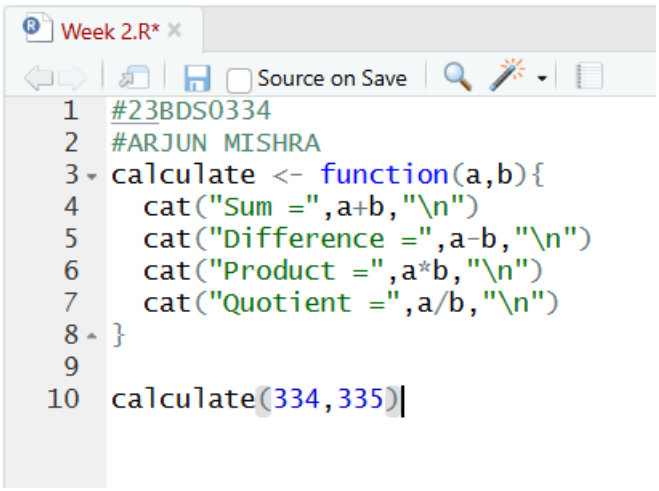
Five built-in mathematical functions were executed successfully.

15. Define a user-defined function that accepts two numbers and returns their sum, difference, product, and quotient.

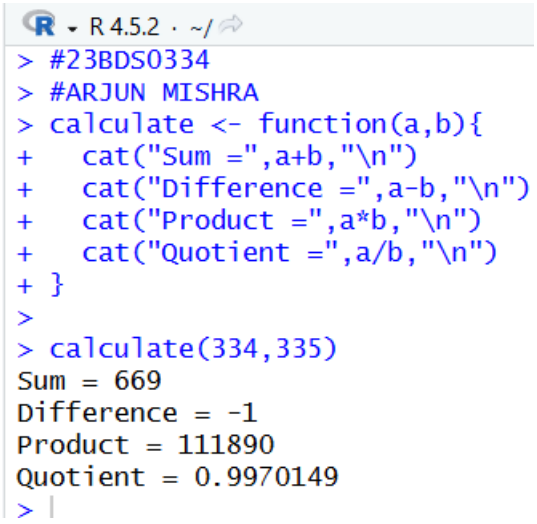
AIM:

To define a user-defined function that returns the sum, difference, product, and quotient of two numbers.

CODE:



OUTPUT:



RESULT:

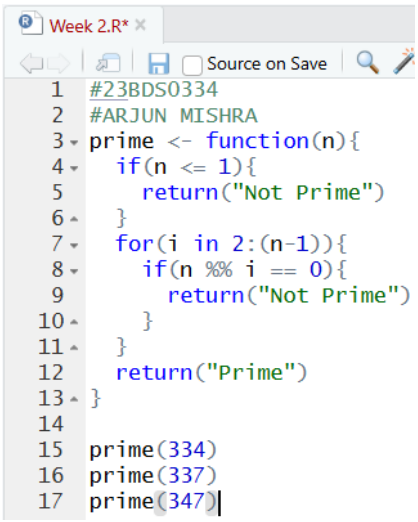
A user-defined function was created successfully to perform basic arithmetic operations.

16. Create a user-defined function to determine whether a given number is prime. Test the function using at least three different numbers.

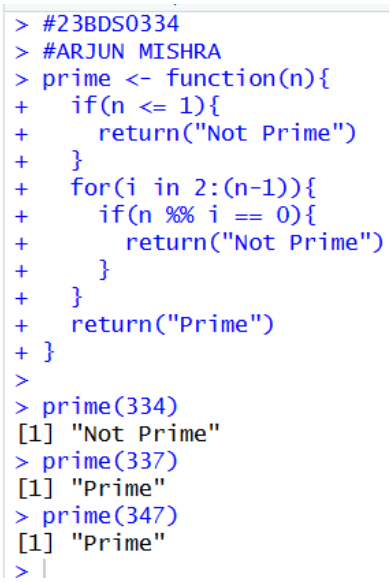
AIM :

To create a user-defined function to check whether a given number is prime and test it with three numbers.

CODE:



OUTPUT:



RESULT:

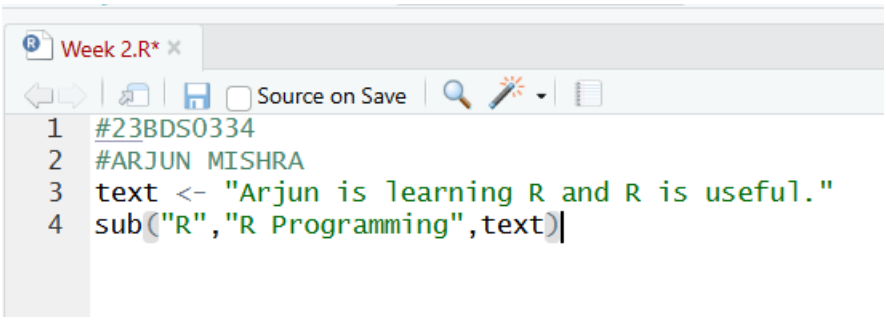
The prime checking function was created successfully and tested with three numbers.

17. Write an R program using the sub() function to replace the first occurrence of the word "R" with "R Programming" in a given character string.

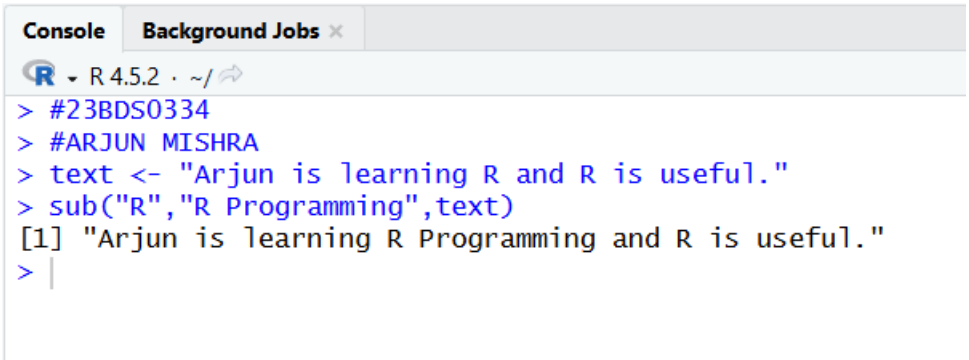
AIM:

To replace the first occurrence of the word "R" with "R Programming" using the sub() function.

CODE:



OUTPUT:



RESULT:

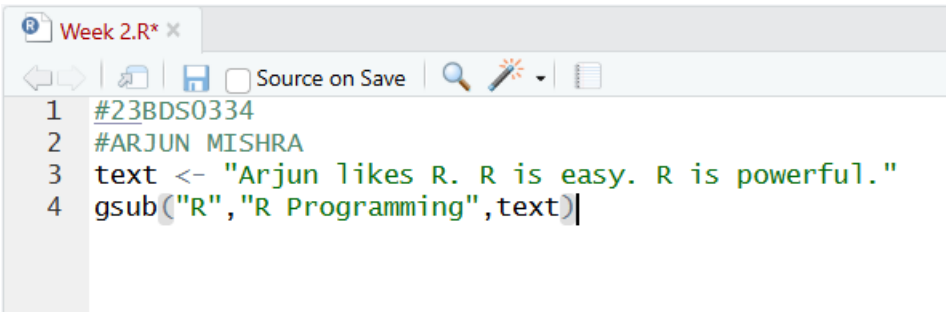
The first occurrence of **"R"** was replaced successfully using sub().

18. Write an R program using the gsub() function to replace all occurrences of a specified word or character in a given sentence.

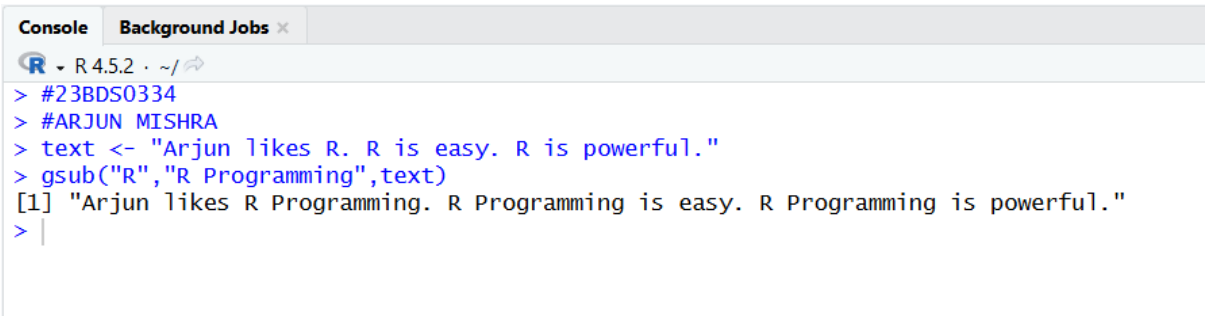
AIM:

To replace all occurrences of a specified word using the gsub() function.

CODE:



OUTPUT:



RESULT:

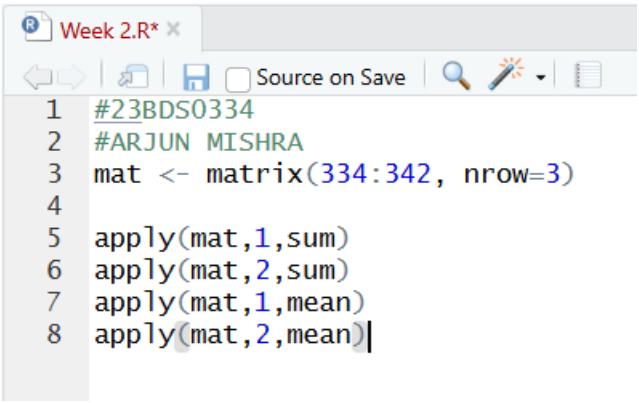
All occurrences of **"R"** were replaced successfully using gsub().

19. Create a matrix containing numeric values and use the apply() function to calculate the sum and mean of each row and each column.

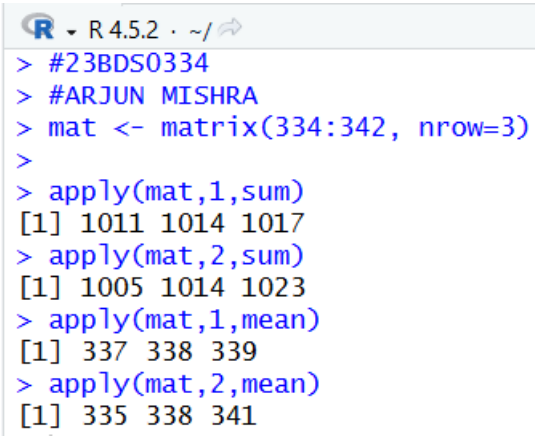
AIM:

To create a matrix and use the apply() function to calculate the sum and mean of each row and column.

CODE:



OUTPUT:



RESULT:

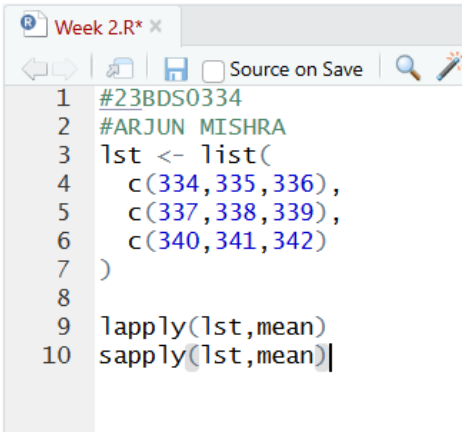
The sum and mean of each row and column were calculated successfully using apply().

20. Create a list containing three numeric vectors and use lapply() and sapply() to calculate the mean of each vector. Compare the formats of the outputs returned by the two functions.

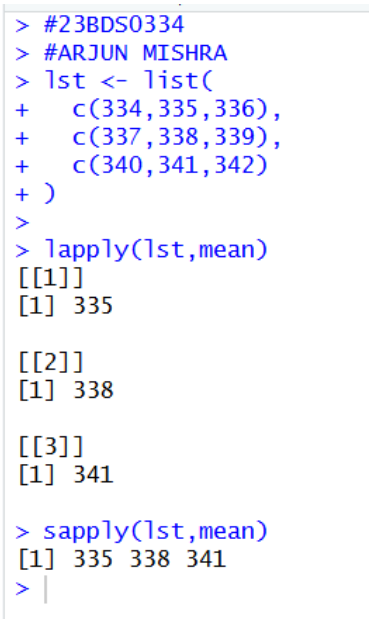
AIM:

To create a list containing three numeric vectors and calculate the mean of each vector using lapply() and sapply().

CODE:



OUTPUT:



RESULT:

The means of all vectors were calculated successfully using both lapply() and sapply(). lapply() returned a list, while sapply() returned a simplified numeric vector.